Formar
Servir
Cuidar



GEMELO DIGITAL DE UN E-COMMERCE

Simulación de eventos discretos bajo campañas de descuento

Modelos y Simulación · Ingeniería de Software
Grupo 1 · Entrega Final · Noviembre de 2026
Ana Isabel López Durán — modelo de simulación y hallazgos
Gerónimo Álvarez Saldarriaga — sistema real y mediciones
Brandon Steven Vidal Jaramillo — validación analítica
github.com/Analopez0604/pa-modelos-simulacion-ecommerce

VIGILADA Mineducación
código GCO- FR-03
Versión: 005
SC 7134-1
SA-CER901605
Tradición - Transformación - Innovación
Alcaldía de Medellín
Distrito de
Ciencia, Tecnología e Innovación

INSTITUCIÓN UNIVERSITARIA PASCUAL BRAVO
Acreditados en Alta Calidad

Formar Servir Cuidar

DIEZ MINUTOS QUE **ROMPEN EL SISTEMA**

×20

la navegación: de 9 a 180 lecturas/s

×6

las compras: de 1 a 6 compras/s

10

minutos que dura la oferta relámpago

Qué se satura y por qué el autoescalado llega tarde

Fluyen peticiones; esperan en la cola de CPU y en la de órdenes. Un nodo nuevo tarda de 2 a 8 min en arrancar y la oferta dura 10.

Lo que el sistema debe cumplir

Indicador	Umbral
p95 de latencia	< 2,0 s
Peticiones no atendidas	< 10 %
Abandono de compras	< 5 %
CPU máxima	< 85 %
Amplificación	< 1,5

P1 · Umbral

¿Bajar el umbral de 80 % a 60 %?

P2 · Timeout

¿Timeout de 2 s, 5 s o 10 s?

P3 · Anticipación

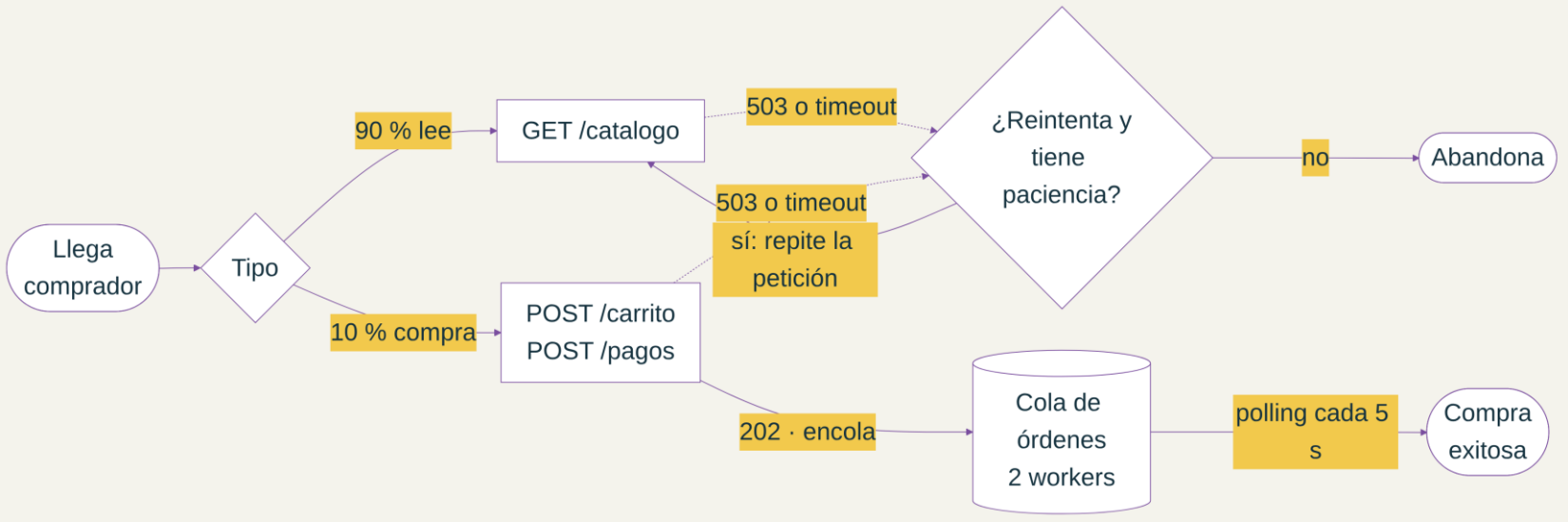
¿Pre-escalar antes? ¿A cuántos nodos?

P4 · Capacidad

¿Hasta cuántos compradores/s aguanta?

Escenario, SLA y preguntas de decisión definidos en la Entrega 1 · informe §1 y §2.1

DEL SISTEMA REAL **AL MODELO DES**



Corre en Docker: FastAPI con 0,5 CPU, PostgreSQL y 2 workers · una fila por petición, más de 78 000 en datos/telemetry_log.csv · código en simulacion/modelo.py

INSTITUCIÓN UNIVERSITARIA PASCUAL BRAVO
Acreditados en Alta Calidad

Formar Servir Cuidar

LO QUE NOS SEÑALÓ **Y CÓMO QUEDÓ**

Subsanación de la Entrega 1

Observación en la E1	Corrección en la final
Solo existía el README	API en Docker, SimPy y contraste M/M/c
Ruta de escritura incompleta	carrito → pago (202) → polling 5 s
Lognormales incompletas	μ y σ por tipo de petición
Paciencia Exponencial	Weibull k = 2
1 worker: ρ = 1,65	2 workers: ρ = 0,83

Parámetros y distribuciones calibradas

Parámetro	Distribución	Fuente
Llegadas	Poisson no homogénea	Entrega 1
Servicio lectura	Lognormal(−5,158; 0,530) media 6,6 ms · desv. 3,8 ms	Medido
Servicio pago	Lognormal(−4,446; 0,534) media 13,5 ms · desv. 7,8 ms	Medido
Procesar orden	Uniforme(0,15; 0,40) s	Worker (media medida 0,28 s)
Arranque de nodo	Uniforme(120; 480) s	Entrega 1
Paciencia	Weibull(k = 2; escala 22,57)	E1 corregida

No modelamos el pool de conexiones: la espera fue de **0,1 a 0,2 ms** y nunca limita · tabla completa en el informe §2.2

INSTITUCIÓN UNIVERSITARIA PASCUAL BRAVO
Acreditados en Alta Calidad

Formar Servir Cuidar

EL MODELO COINCIDE CON **M/M/c**

Pusimos el mismo modelo en modo M/M/c (llegadas Poisson, servicio Exponencial, cola infinita, sin timeout ni abandono) y lo comparamos con Erlang C. λ = 135, ρ = 0,9.

Métrica	M/M/c	SimPy	Error
Utilización ρ	0,9000	0,8992	0,1 %
Cola media Lq	6,661	6,650	0,2 %
Espera en cola Wq	49,34 ms	49,26 ms	0,2 %
Tiempo total W	89,34 ms	89,26 ms	0,1 %

Con λ = 105 y 120 el error máximo es **2,4 %**. La **ley de Little** se cumple: L medido y λ · W difieren menos del 0,5 % en los cuatro casos.

Eso, más la oferta que cambia en el tiempo y los minutos que tardan los nodos en arrancar, es lo que la fórmula no puede representar.

¿Y entonces para qué simular?

Con ρ = 1,2, M/M/c no tiene solución

La cola crece sin parar: a los 30 min la espera ya va en 205 segundos.

El sistema real sí se estabiliza

Con tope de acceso, timeout, reintentos y abandono se queda en W = 0,66 s… rechazando el 38 %.

30 réplicas de 30 min · **analisis/contraste_mmc.py** · informe §4.1 **y docs/contraste_mmc.md**

INSTITUCIÓN UNIVERSITARIA PASCUAL BRAVO
Acreditados en Alta Calidad

Formar Servir Cuidar

HOY SE PIERDE **1 DE CADA 5 COMPRADORES**

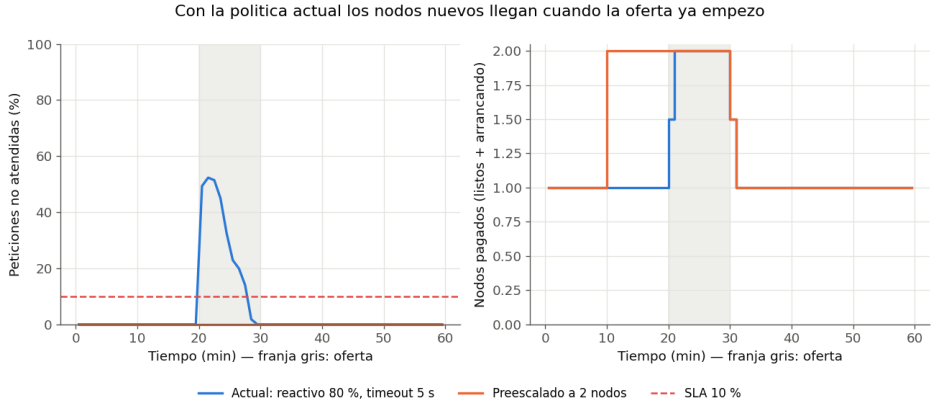
Política actual en los 10 min de la oferta · reactivo al 80 %, timeout 5 s · media ± IC 95 %, 30 réplicas

33,0 %
de las peticiones no se atiende
± 3,5 · SLA: < 10 %

20,1 %
de los compradores abandona
± 2,8 · SLA: < 5 %

100 %
de CPU máxima durante la oferta
SLA: < 85 %

Cuidado con el p95: dice que todo va bien
Cumple el SLA, pero solo mide lo que sí se atendió. Se lee junto con las no atendidas.



Evolución minuto a minuto de la hora completa. La caída está concentrada en los **10 minutos de la oferta**: antes y después el sistema va holgado. El segundo nodo llega cuando la ola ya pasó.

Causa: un nodo nuevo tarda de 2 a 8 min en arrancar · docs/figuras/sim_evolucion_campana.png · informe §5.1

INSTITUCIÓN UNIVERSITARIA PASCUAL BRAVO
Acreditados en Alta Calidad

Formar Servir Cuidar

NI EL UMBRAL NI EL TIMEOUT **CAMBIAN NADA**

P1 · Bajar el umbral de 80 % a 60 %

Mismo resultado, 18 % más caro

La CPU salta al 100 % y con cualquier umbral se pide un nodo.

El daño ocurre mientras ese nodo arranca. Y con 60 % se pide un tercero que llega tarde.

+12,6 nodos·min (+18 %) sin ningún beneficio

P2 · Cambiar el timeout: 2 s, 5 s o 10 s

Los tres dan exactamente lo mismo

100 peticiones como máximo × 7,4 ms de CPU cada una:

100 × 7,4 ms ≈ 0,74 s

Justo el p95 medido. El nodo rechaza antes que el timeout.

30 réplicas por configuración · informe §5.2, preguntas de decisión P1 y P2

INSTITUCIÓN UNIVERSITARIA PASCUAL BRAVO
Acreditados en Alta Calidad

Formar Servir Cuidar

LO QUE SÍ FUNCIONA: **LLEGAR ANTES**

P3 · Pedir los nodos **10 minutos antes** de que arranque la oferta, en vez de esperar a que la CPU suba.

Indicador	SLA	Política actual	Pre-escalar a 2	Pre-escalar a 3
Peticiones no atendidas	< 10 %	33,0 %	0,0 %	0,0 %
Abandono de compras	< 5 %	20,1 %	0,0 %	0,0 %
CPU máxima	< 85 %	100 %	73 %	49 %
p95 de latencia	< 2,0 s	0,74 s	0,03 s	0,02 s
p99 de latencia	—	0,78 s	0,04 s	0,03 s
Costo de la hora	comparar	70,0	80,5 (+15 %)	105,0 (+50 %)

El canje: Pagar 10 minutos más de un nodo por campaña para no perder a uno de cada cinco compradores. Con 3 nodos también se cumple, pero cuesta 50 % más y no mejora ni un indicador.

Costo en nodos·min de la hora completa · 30 réplicas · informe §5.1 y §5.2

INSTITUCIÓN UNIVERSITARIA PASCUAL BRAVO
Acreditados en Alta Calidad

Formar Servir Cuidar

EL LÍMITE FÍSICO Y **DÓNDE SE MUEVE EL CUELLO**

Límite físico duro (C6) · medido, no supuesto

7,4 ms

de CPU por petición (± 0,8), despejado con la ley de utilización **D = U / X** en las 7 etapas sin rechazos.

→ un nodo no pasa de 135 peticiones/s

El modelo acierta donde se puede comprobar

A 100 compradores/s el modelo predice **10,6 %** y el nodo real dio **10,6 %.**

Sensibilidad: ¿hasta dónde aguanta?

Compradores/s	1 nodo	2 nodos
90	0,9 / 96	1,0 / 49
100	10,6 / 100	3,4 / 55
120	36,5 / 100	6,9 / 66
140	51,2 / 100	9,4 / 77
160	60,1 / 100	11,6 / 88

Pasando 140 cambia el cuello de botella

La CPU no se satura (88 % a 160) y aun así se cae el 11,6 %: el límite son los 2 workers.

Cada celda: no atendidas (%) / CPU (%) · 10 réplicas por punto · SLA: no atendidas < 10 % · informe §3.1 y §5.2

INSTITUCIÓN UNIVERSITARIA PASCUAL BRAVO
Acreditados en Alta Calidad

Formar Servir Cuidar

LO QUE **RECOMENDAMOS**

Pre-escalar a 2 nodos 10 minutos antes de cada oferta

Y dejar el autoescalado reactivo al 80 % activo como respaldo: es el mismo que devuelve la capacidad a un nodo cuando la campaña termina.

Lo que gana

Abandono de 20 % a 0 % y CPU máxima de 100 % a 73 %.

Lo que cuesta

15 % más por hora: 80,5 contra 70,0 nodos·min.

Lo que no hay que hacer

Bajar el umbral (18 % más caro) ni tocar el timeout.

El techo

Pasando 140 comp/s, sumar nodos y workers.

Hasta dónde llega el modelo: Supone la campaña programada · no modela caídas de nodos, varias regiones ni ataques · el abandono no distingue su causa.

INSTITUCIÓN UNIVERSITARIA PASCUAL BRAVO
Acreditados en Alta Calidad

Formar Servir Cuidar

DEMOSTRACIÓN **EN VIVO**

1 · Levantar

```
docker compose up --build -d
```

API, base de datos y 2 workers

2 · Cargar · 70 s

```
python sistema_real/gateway.py --escenario pico --duracion 10
```

La CPU sube y salen los 503

3 · Telemetría

```
tail -f datos/telemetry_log.csv
```

Una fila por petición, en vivo

4 · Simular · 1-2 min

```
python simulacion/main.py --rapido
```

SimPy en consola, 5 réplicas

Qué se ve

Que el sistema real se satura como el modelo lo predijo.

Al terminar

`git restore datos/ y git clean -f datos/`

El panel (streamlit run panel.py) solo si sobra tiempo · respaldo: corridas guardadas en docs/resultados_simulacion.md

INSTITUCIÓN UNIVERSITARIA
PASCUAL BRAVO®
Acreditados en Alta Calidad

Formar
Servir
Cuidar



¡ GRACIAS !

Tradición - Transformación - Innovación

¿Preguntas?

Ana Isabel López Durán · Gerónimo Álvarez Saldarriaga · Brandon Steven Vidal Jaramillo
Modelos y Simulación · Grupo 1 · I.U. Pascual Bravo

Alcaldía de Medellín
Distrito de
Ciencia, Tecnología e Innovación

github.com/Analopez0604/pa-modelos-simulacion-ecommerce